Sistema de gestión de estudiantes básico

Estudiante:

Andrés Felipe Gómez Rios

Materia y grupo:

Algoritmos y programación 2310-1A

Fecha de Entrega:

7/11/2026

Instructor:

Eliana Marcela Paez Perdomo

Objetivo:

Programación de una aplicación de gestión de estudiantes básico, que cumpla con las siguientes funciones:

- Agregar estudiante
- Listar estudiantes
- Buscar estudiantes por ID

Explicación del problema que resuelve el software

Organiza la información de estudiantes, con sus respectivas propiedades, nombre, apellido, ID y semestre, para de esta manera tener una base de datos de la cual se podrá utilizar la información como convenga.

Tecnologías y lenguaje de programación utilizados

- Para el flujograma se utilizó la aplicación web lucid chart
- Como IDE se utilizó visual estudio code
- Como lenguaje de programación se utilizó Python.

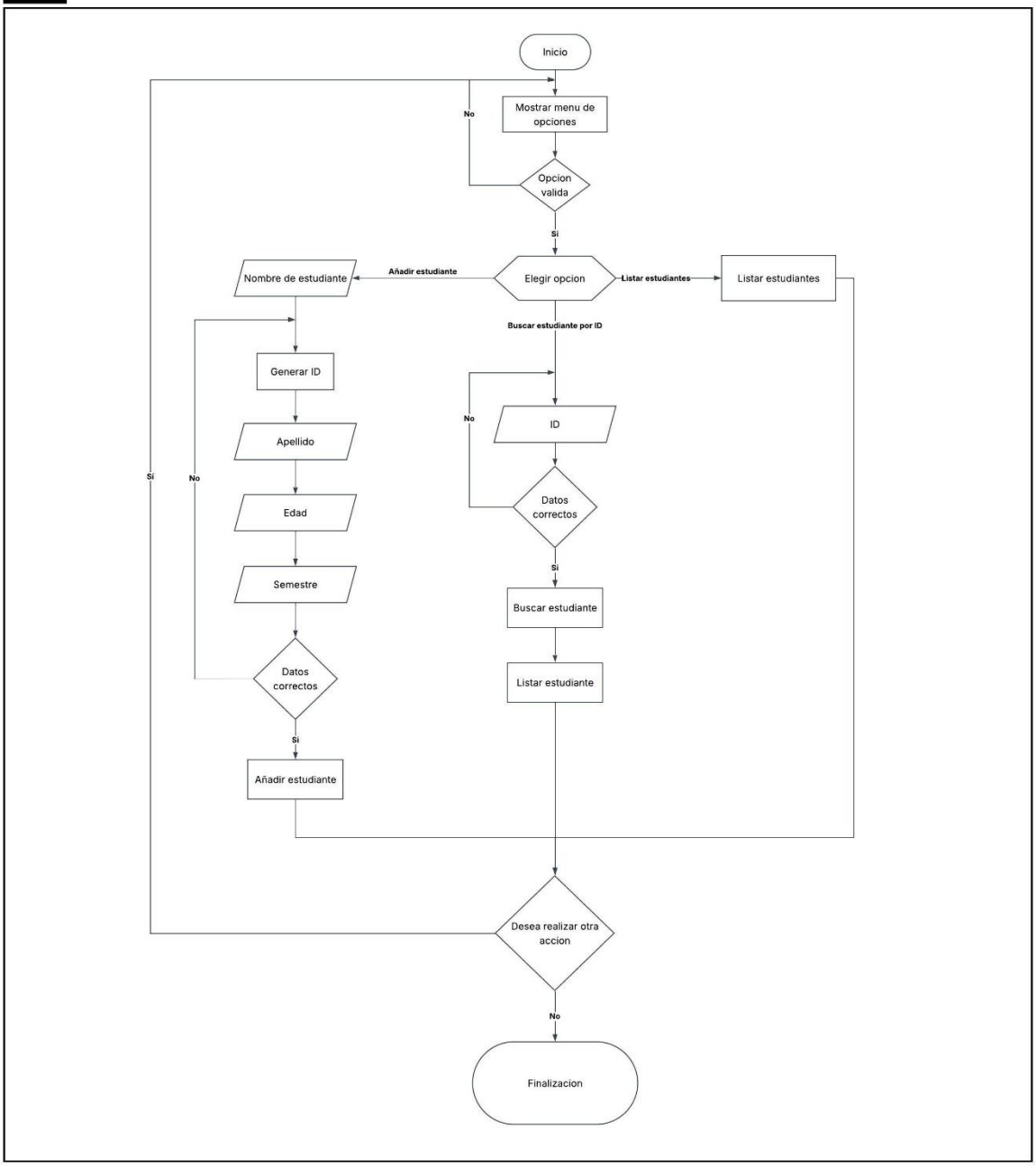
Descripción general de la solución implementada

Programa de gestión de estudiantes, el cual puede crear nuevos estudiantes y almacenarlos en formato JSON, puede listar los estudiantes totales y realizar consultas por ID.

El programa funciona en consola.

Diagrama de flujo

Marco 1



Se utilizan óvalos para definir inicio y fin, rectángulos para definir cada paso, paralelogramos para definir inputs y rombos para definir decisiones.

Análisis de estructuras de control

Se utilizan bucles while para revisar la información entregada en los inputs.

Se utilizan bucles if para elegir entre diferentes opciones.

Se utilizan for para recorrer listas, de estudiantes o id.

Codigo fuente

```
# IMPORTACION DE JSON
import json

# MOSTRAR MENU
print("Que quiere hacer?")
print("1. Añadir estudiante")
print("2. Listar estudiantes")
print("3. Buscar estudiante por ID")
print("4. Salir")

# CARGA ESTUDIANTES DE LA BASE DE DATOS DE ESTUDIANTES
def Cargar_Estudiantes():
    try:
        with open("estudiantes.json", "r", encoding="utf-8") as archivo:
            return json.load(archivo)

    except FileNotFoundError:
        return []

    except json.JSONDecodeError:
        return []

lista_estudiantes = Cargar_Estudiantes()

# AÑADE ESTUDIANTES NUEVOS

def Añadir_Estudiante():

    id_estudiante = Generar_Id()
    nombre = Validar_Texto("Ingrese el nombre del estudiante: ")
    apellido = Validar_Texto("Ingrese el apellido del estudiante: ")

    while True:
        try:
            edad = int(input("Ingrese la edad: "))

            if 15 <= edad <= 100:
                break
            else:
                print("La edad debe estar entre 15 y 100")

        except ValueError:
            print("Debe ingresar un número entero")
```

```
    while True:
        try:
            semestre = int(input("Ingrese la semestre: "))

            if 1 <= semestre <= 10:
                break
            else:
                print("La edad debe estar entre 1 y 10")

        except ValueError:
            print("Debe ingresar un número entero")

    estudiante = {
        "id": id_estudiante,
        "nombre": nombre,
        "apellido": apellido,
        "edad": edad,
        "semestre": semestre
    }
    lista_estudiantes.append(estudiante)

    Guardar_Estudiantes()

    print(f"Estudiante Añadido: {estudiante}")

# GENERA ID UNICO

def Generar_Id():
    if not lista_estudiantes:
        return 1

    return max(e["id"] for e in lista_estudiantes) + 1

# FUNCION QUE VALDA QUE UN TEXTO SEA TEXTO

def Validar_Texto(mensaje):
    while True:
        texto = input(mensaje).strip()

        if texto and texto.replace(" ", "").isalpha():
            return texto
        else:
            print("Ingrese un texto válido, sin números.")
```

```python
# FUNCION QUE GUARDA EL ESTUDIANTE EN ARCHIVO JSON

def Guardar_Estudiantes():
    with open("estudiantes.json", "w", encoding="utf-8") as archivo:
        json.dump(lista_estudiantes, archivo, indent=4, ensure_ascii=False)

# FUNCION QUE LISTA ESTUDIANTES

def Listar_Estudiantes():
    for estudiante in lista_estudiantes:
        print(estudiante)

# FUNCION QUE BUSCA ESTUDIANTES POR ID

def Buscar_Estudiante():
    id_list = []
    estudiante = None
    for estudiante in lista_estudiantes:
        estudiante_Id = estudiante["id"]
        id_list.append(estudiante_Id)

    # EVALUA QUE EL ID SOLICITADO SEA VALIDO Y EXISTA

    while True:
        try:
            id = int(input("Escriba el id del estudiante: "))
            if id in id_list:
                break
            else:
                print("registro no encontrado")
        except ValueError:
                    print("Debe ingresar un número entero")

    # RECORRE LISTA DE ESTUIDANTES PARA BUSCAR EL ESTUDIANTE CON EL ID
DESEADO

    for buscado in lista_estudiantes:
        if buscado["id"] == id:
            estudiante = buscado
            break
    return estudiante

# ELIGE OPCIONES Y EVALUA QUE LA OPCION DESEADA SEA VALIDA

while True:
```

```
    try:
        opcion = int(input("Ingrese una opcion: "))

        if 1 <= opcion <=4:
            break
        else:
            print("Elija una opcion entre 1 y 4")
    except ValueError:
        print("Debe ingresar un número entero")


if opcion == 1:
    print("Añadir un estudiante")
    Añadir_Estudiante()
elif opcion == 2:
    print("Listar estudiantes")
    Listar_Estudiantes()
elif opcion == 3:
    print("Buscar estudiante por ID")
    print(Buscar_Estudiante())
elif opcion == 4:
    print("Salir")
```

Evidencias de ejecución

Añadir estudiante

```
Actividad_4
> Python index.py
Que quiere hacer?
1. Añadir estudiante
2. Listar estudiantes
3. Buscar estudiante por ID
4. Salir
Ingrese una opcion: 1
Añadir un estudiante
Ingrese el nombre del estudiante: Andres Felipe
Ingrese el apellido del estudiante: Gomez Rios
Ingrese la edad: 28
Ingrese la semestre: 1
Estudiante Añadido: {'id': 1, 'nombre': 'Andres Felipe', 'apellido': 'Gomez Rios', 'edad': 28, 'semestre': 1}
Actividad_4
>
```

```
Actividad_4
> Python index.py
Que quiere hacer?
1. Añadir estudiante
2. Listar estudiantes
3. Buscar estudiante por ID
4. Salir
Ingrese una opcion: 1
Añadir un estudiante
Ingrese el nombre del estudiante: Juan David
Ingrese el apellido del estudiante: Gomez Rios
Ingrese la edad: 21
Ingrese la semestre: 1
Estudiante Añadido: {'id': 2, 'nombre': 'Juan David', 'apellido': 'Gomez Rios', 'edad': 21, 'semestre': 1}
Actividad_4
>
```


```
[
    {
        "id": 1,
        "nombre": "Andres Felipe",
        "apellido": "Gomez Rios",
        "edad": 28,
        "semestre": 1
    },
    {
        "id": 2,
        "nombre": "Juan David",
        "apellido": "Gomez Rios",
        "edad": 21,
        "semestre": 1
    }
]
```

Listar estudiantes

```
Actividad_4
> Python index.py
Que quiere hacer?
1. Añadir estudiante
2. Listar estudiantes
3. Buscar estudiante por ID
4. Salir
Ingrese una opcion: 2
Listar estudiantes
{'id': 1, 'nombre': 'Andres Felipe', 'apellido': 'Gomez Rios', 'edad': 28, 'semestre': 1}
{'id': 2, 'nombre': 'Juan David', 'apellido': 'Gomez Rios', 'edad': 21, 'semestre': 1}
Actividad_4
>
```

Buscar estudiante por ID

```
Actividad_4
> Python index.py
Que quiere hacer?
1. Añadir estudiante
2. Listar estudiantes
3. Buscar estudiante por ID
4. Salir
Ingrese una opcion: 3
Buscar estudiante por ID
Escriba el id del estudiante: 1
{'id': 1, 'nombre': 'Andres Felipe', 'apellido': 'Gomez Rios', 'edad': 28, 'semestre': 1}
Actividad_4
>
```

Salir

```
Actividad_4
> Python index.py
Que quiere hacer?
1. Añadir estudiante
2. Listar estudiantes
3. Buscar estudiante por ID
4. Salir
Ingrese una opcion: 4
Salir
Actividad_4
>
```

Análisis y reflexión

Se cumplen los 4 objetivos, controlando errores de input en textos y números, se pueden añadir estudiantes, listar estudiantes y buscar estudiantes por ID.

Adicionalmente se implementó un sistema de almacenamiento de información estructurada, mediante JSON.

Durante el desarrollo, se encontraron dificultades a la hora de crear un sistema que funcionara como base de datos, para el cual se escogió el archivo JSON, y se investigo como funciona para poder obtener y crear información de los archivos.

Durante el ejercicio se aprendió a organizar un flujograma y plasmarlo como código. Se aprendió a implementar bucles while y try para repetir un input cuando no se satisface la opción o tipo de dato.

Se aprendió a crear una base de datos muy básica por medio de JSON.

Las estructuras de control utilizadas se decidieron según su funcionamiento y los beneficios que se podían aplicar a cada caso.

Para repetir cada uno de los inputs, cuando no se satisfacen los requerimientos se usaron bucles while ya que al no devolver un True, permite volver a ejecutar fragmentos de código en blucle.

Para recorrer las listas de estudiantes y de ID de los estudiantes se uso el bucle for, ya que este permite observar cada uno de los elementos en una lista.

Para tomar decisiones, se utilizo el bucle if, ya que esta permite tomar caminos diferentes dependiendo de el input.